Table of contents

Stochastic Validation with Gillespie

Simon Frost 2026-05-13

Introduction

Deterministic models — whether individual-based (order 1) or pair-based (order 2) — are approximations of the true stochastic process that governs disease spread on a network. They replace integer-valued node states with continuous probabilities and assume that correlations beyond a certain order can be closed. These approximations work well for large populations far from the epidemic threshold, but they can be misleading when stochastic effects dominate: early in an outbreak, near the critical threshold, or in small populations.

The Gillespie algorithm (Gillespie, 1977) provides an exact stochastic simulation of the continuous-time Markov chain on a specific graph. Each event — an infection along an edge or a recovery of a node — is drawn from the correct waiting-time distribution. NodeBasedModels implements this using JumpProcesses.jl with `ConstantRateJump` events and the `Direct` aggregator.

In this vignette we:

1. Run single stochastic trajectories and visualise the step-function dynamics
2. Explore stochastic variability across runs
3. Compute ensemble averages with confidence bands
4. Compare the Gillespie ground truth with deterministic approximations
5. Examine the effect of initial conditions on stochastic extinction
6. Inspect the bimodal final-size distribution

Setup

```
using NodeBasedModels
using Graphs
using Plots
using Statistics
```

Single stochastic trajectory

We start with a random regular graph where every node has degree 6. This is a natural test case because the homogeneous degree structure means population-level theory should apply, and deviations are purely due to stochasticity and local structure.

```
g = random_regular_graph(100, 6; seed=42)
net = GraphNetwork(g)
println("Nodes: ", nv(g), ", Edges: ", ne(g), ", Mean degree: ", round(mean_degree(net), d
```

```
Nodes: 100, Edges: 300, Mean degree: 6.0
```

Run a single Gillespie SIR simulation starting from one infected node:

```
result = gillespie_sir(net;
    infection_rate=0.125,
    recovery_rate=0.25,
    initial_infected=[1],
    seed=1
)
```

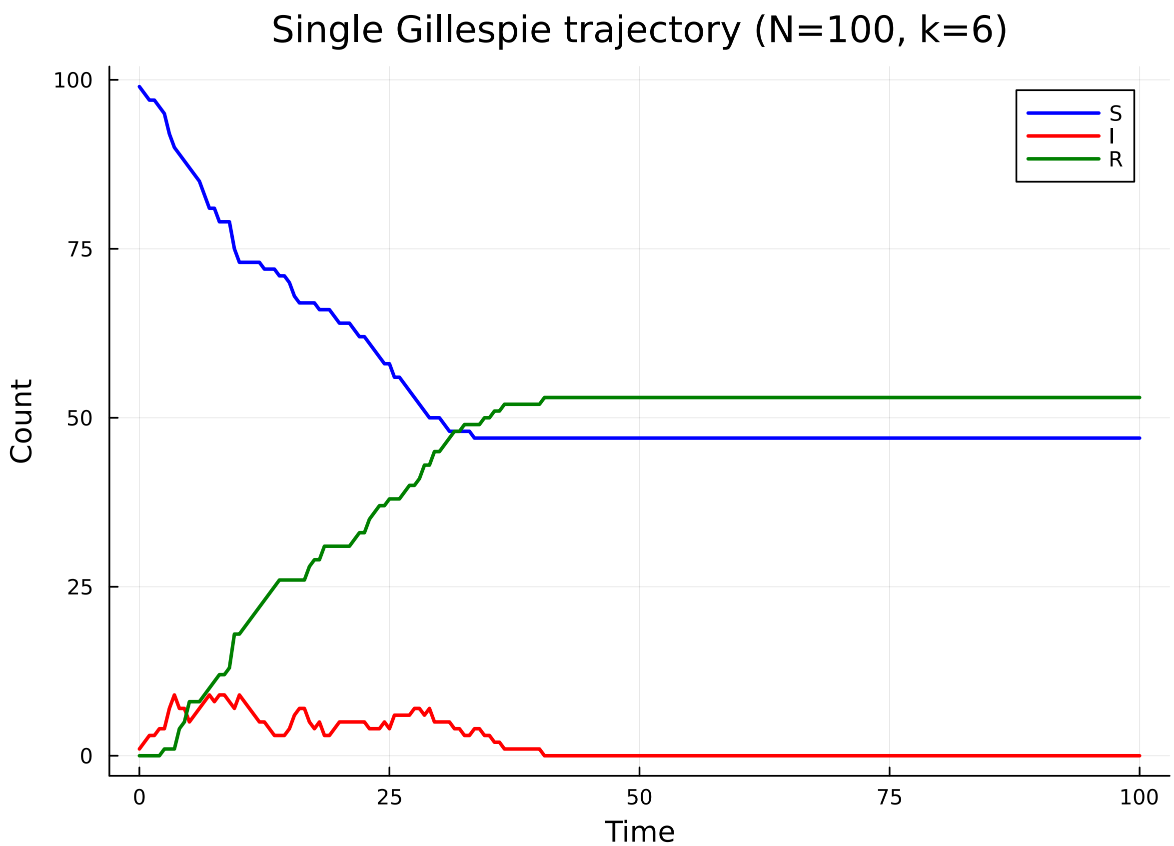
```
GillespieResult(N=100, tspan=(0.0, 100.0))
```

The `aggregate` function with `saveat` interpolates the step-function trajectory onto a regular time grid:

```julia
S_agg = aggregate(result, :S; saveat=0.5)
I_agg = aggregate(result, :I; saveat=0.5)
R_agg = aggregate(result, :R; saveat=0.5)
```

```
(times = [0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5  …  95.5, 96.0, 96.5, 97.0, 97.
```

```julia
plot(S_agg.times, S_agg.counts, label="S", lw=2, color=:blue,
     xlabel="Time", ylabel="Count", title="Single Gillespie trajectory (N=100, k=6)")
plot!(I_agg.times, I_agg.counts, label="I", lw=2, color=:red)
plot!(R_agg.times, R_agg.counts, label="R", lw=2, color=:green)
```

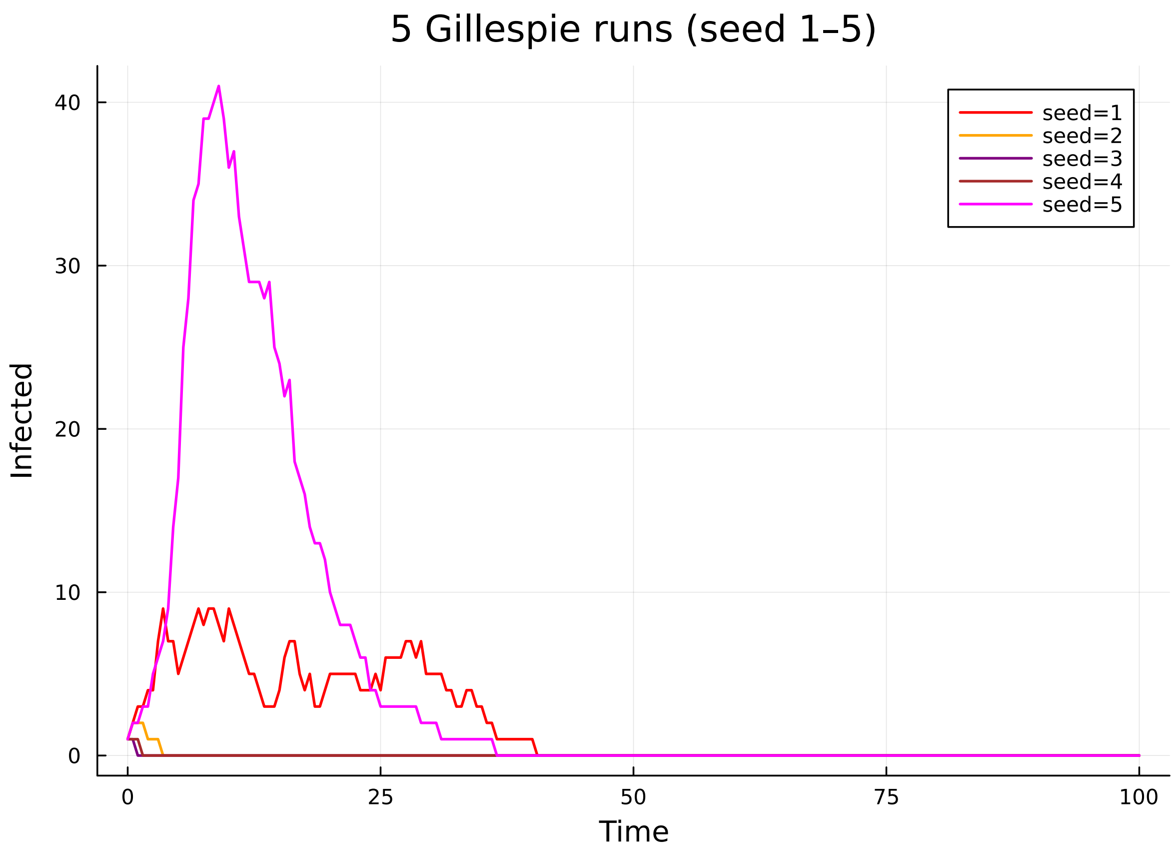


The step-function nature of the trajectory reflects the discrete state changes: each jump corresponds to exactly one infection or recovery event.

Stochastic variability

A single trajectory tells us what *could* happen but not what *typically* happens. Let us run 5 simulations with different random seeds to see the range of outcomes:

```
p = plot(xlabel="Time", ylabel="Infected", title="5 Gillespie runs (seed 1–5)")
colors = [:red, :orange, :purple, :brown, :magenta]
for (idx, s) in enumerate(1:5)
    res = gillespie_sir(net;
        infection_rate=0.125,
        recovery_rate=0.25,
        initial_infected=[1],
        seed=s
    )
    I_s = aggregate(res, :I; saveat=0.5)
    plot!(p, I_s.times, I_s.counts, label="seed=$s", lw=1.5, color=colors[idx])
end
p
```



Some runs produce large epidemics while others die out quickly — this is stochastic extinction, where the initial infected node recovers before passing the infection on. The probability of extinction is especially high when starting from a single infected individual.

Ensemble averaging

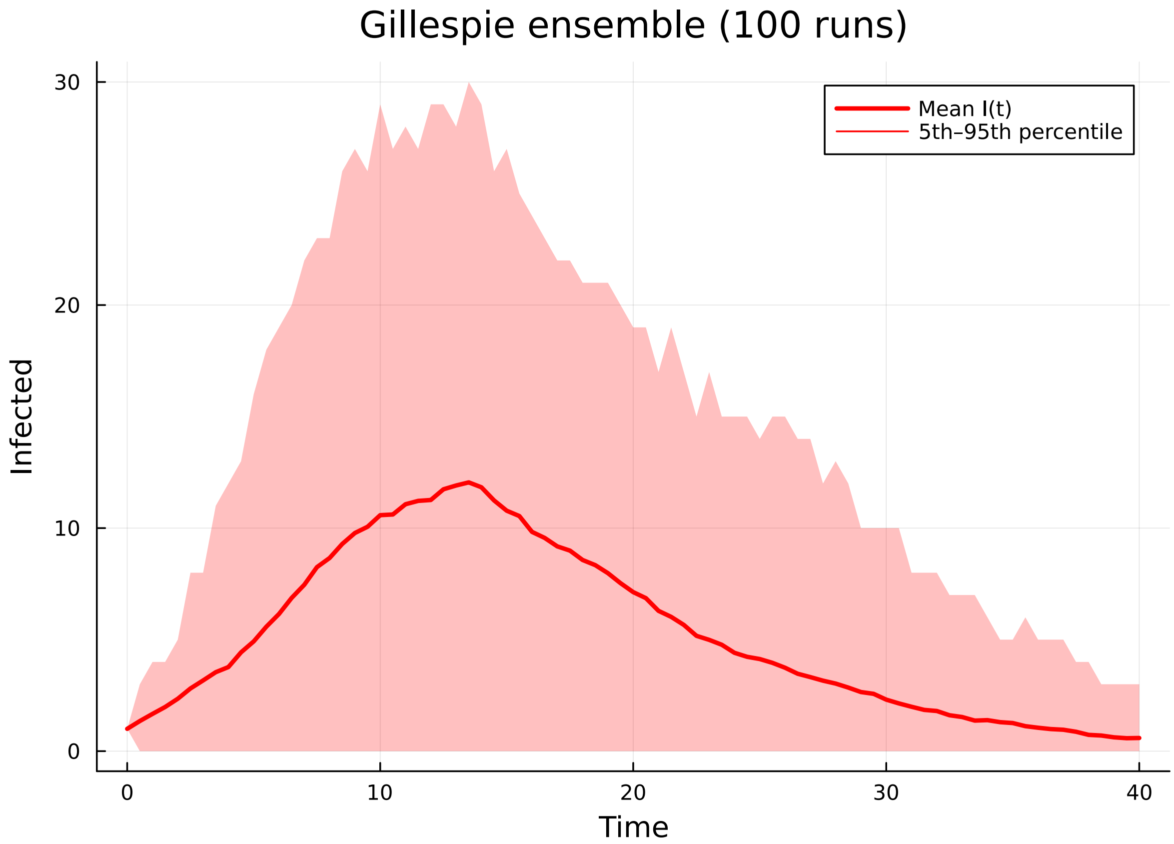
To characterise the typical behaviour we run many simulations and compute summary statistics. The `gillespie_sir_average` function automates this:

```
ens = gillespie_sir_average(net;
    nruns=100,
    dt=0.5,
    tmax_grid=40.0,
    infection_rate=0.125,
    recovery_rate=0.25,
    initial_infected=[1]
)
```

```
(t_grid = [0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5  …  35.5, 36.0, 36.5, 37.0, 37
```

Plot the mean infected curve with a ribbon showing the 5th–95th percentile range:

```
plot(ens.t_grid, ens.I_mean, label="Mean I(t)", lw=2.5, color=:red,
     xlabel="Time", ylabel="Infected",
     title="Gillespie ensemble (100 runs)")
plot!(ens.t_grid, ens.I_mean, ribbon=(ens.I_mean .- ens.I_q05, ens.I_q95 .- ens.I_mean),
      fillalpha=0.25, color=:red, label="5th–95th percentile")
```



The wide ribbon at early times reflects the stochastic extinction events. After the epidemic establishes, the trajectories converge and the ribbon narrows.

Comparison with deterministic models

Now we overlay the individual-based (order 1) and pair-based (order 2) deterministic approximations on the Gillespie envelope:

```
ib = generate_individual_based(sir_model(), net;
    infection_rate=0.125,
    recovery_rate=0.25,
    initial_infected=[1],
    tspan=(0.0, 40.0),
    saveat=0.5,
)

pb = generate_pair_based(sir_model(), net;
```

```
    infection_rate=0.125,
    recovery_rate=0.25,
    initial_infected=[1],
    tspan=(0.0, 40.0),
    saveat=0.5,
)
```

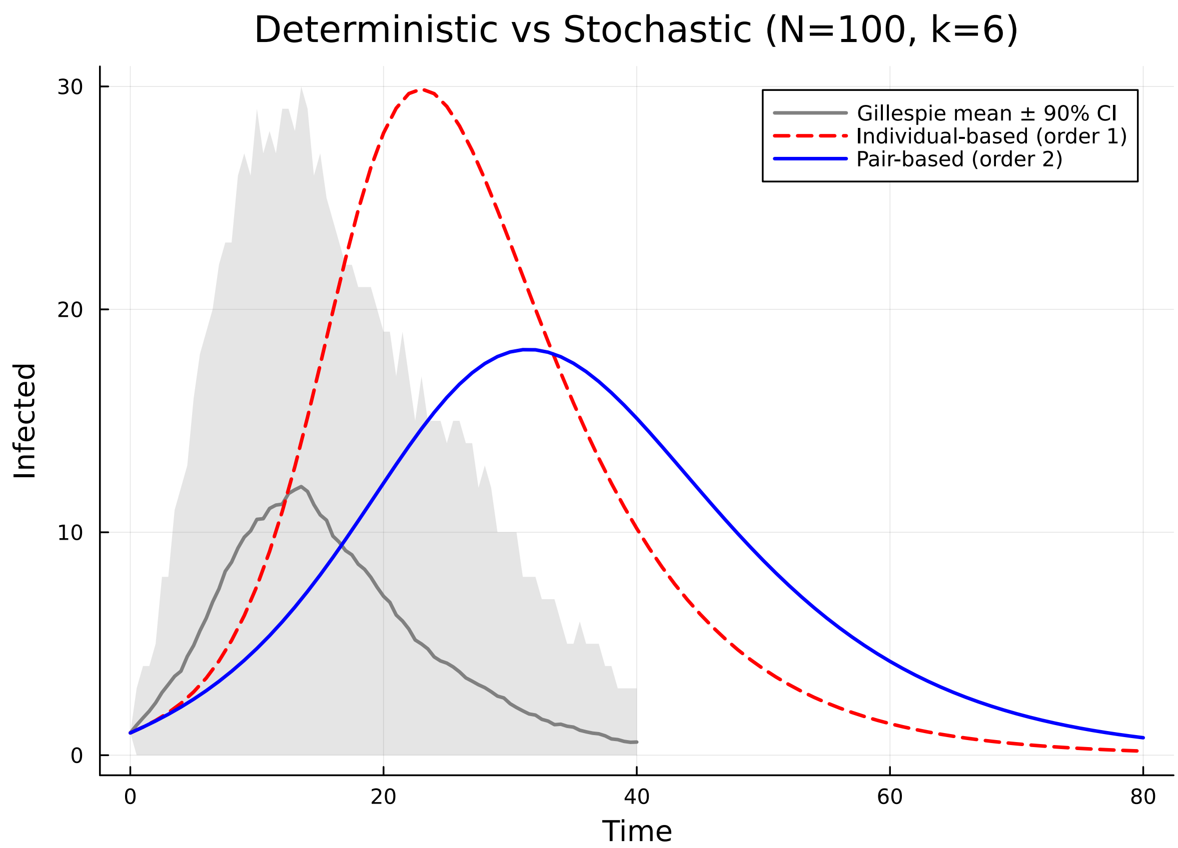
```
PairBasedResult(N=100, edges=600, tspan=(0.0, 40.0))
```

(Both constructors accept either `initial_infected` for an exact seed set or `ε`/`seed_fraction` for a uniform low-prevalence seed; the two options are mutually exclusive — `initial_infected` takes precedence when both are supplied.)

```
I_ib = aggregate(ib, :I)
I_pb = aggregate(pb, :I)
t_det = range(0.0, 80.0, length=length(I_ib))
```

```
0.0:1.0:80.0
```

```
plot(ens.t_grid, ens.I_mean,
     ribbon=(ens.I_mean .- ens.I_q05, ens.I_q95 .- ens.I_mean),
     fillalpha=0.2, color=:grey, lw=2, label="Gillespie mean ± 90% CI",
     xlabel="Time", ylabel="Infected",
     title="Deterministic vs Stochastic (N=100, k=6)")
plot!(t_det, I_ib, label="Individual-based (order 1)", lw=2, ls=:dash, color=:red)
plot!(t_det, I_pb, label="Pair-based (order 2)", lw=2, color=:blue)
```



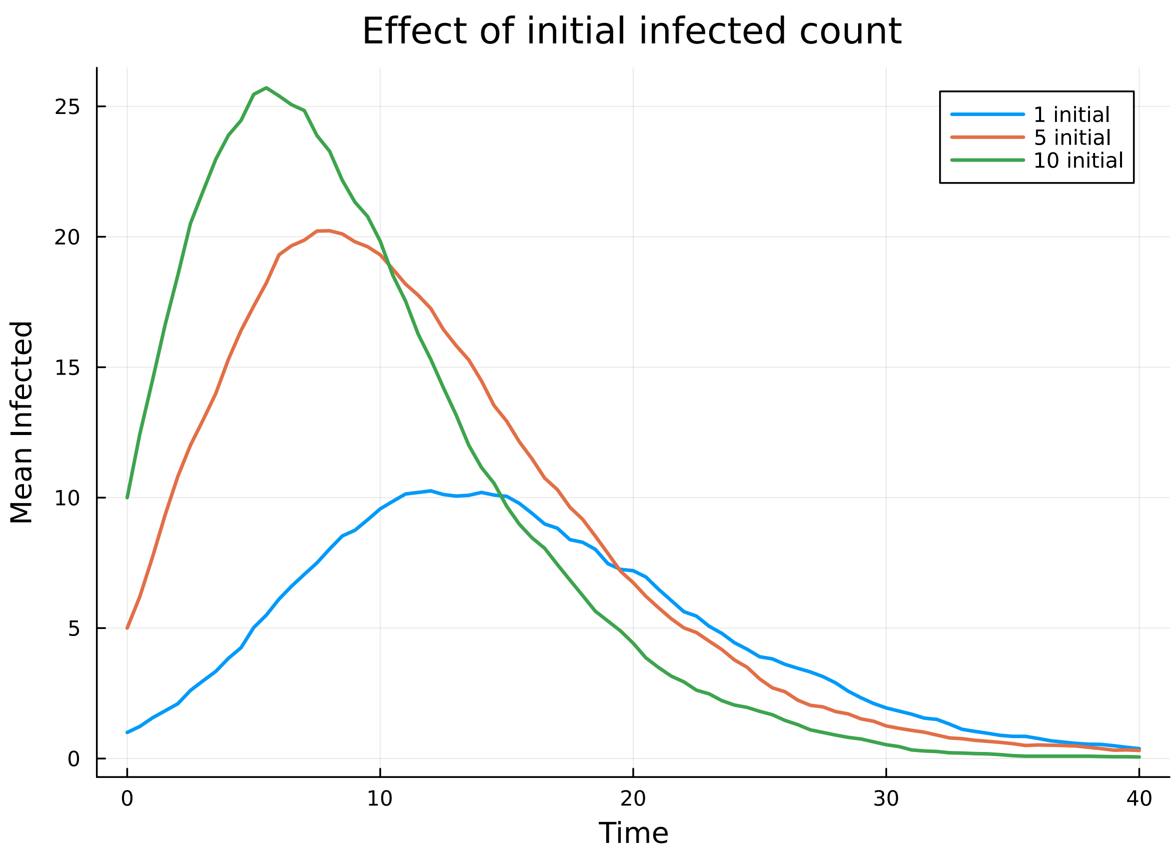
Key observations:

- The individual-based model (dashed red) tends to overestimate the epidemic because it ignores pair correlations — it assumes neighbours are infected independently, overpredicting transmission.
- The pair-based model (solid blue) tracks closer to the Gillespie mean because it captures the dynamical correlations between connected nodes.
- The Gillespie confidence band shows the range of stochastic uncertainty that no deterministic model can capture.

Effect of initial conditions

Stochastic extinction probability depends strongly on how many nodes are initially infected. With a single seed, the epidemic must survive a “bottleneck”; with more seeds, it is far more likely to establish.

```julia
p = plot(xlabel="Time", ylabel="Mean Infected",
         title="Effect of initial infected count")
for n_init in [1, 5, 10]
    init_nodes = collect(1:n_init)
    ens_init = gillespie_sir_average(net;
        nruns=100,
        dt=0.5,
        tmax_grid=40.0,
        infection_rate=0.125,
        recovery_rate=0.25,
        initial_infected=init_nodes
    )
    plot!(p, ens_init.t_grid, ens_init.I_mean,
          label="$n_init initial", lw=2)
end
p
```

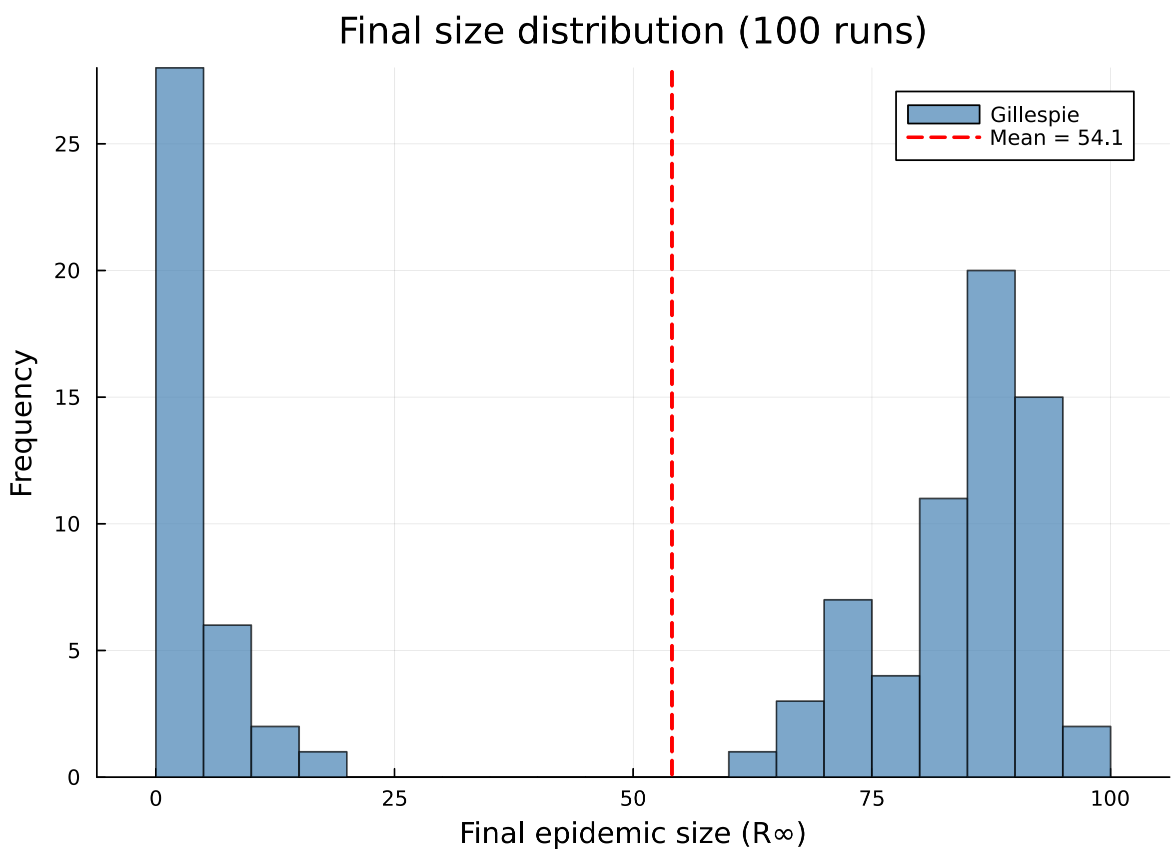


With 10 initial infections the epidemic almost always takes off, producing a higher and earlier peak in the mean curve. With 1 initial infection many runs die out, pulling down the average.

Final size distribution

The `final_sizes` field from `gillespie_sir_average` records the total number of recovered individuals at the end of each run. This distribution is typically bimodal: one mode near zero (stochastic extinction) and one near the deterministic final size (major outbreak).

```
histogram(ens.final_sizes, bins=20,
    xlabel="Final epidemic size (R∞)",
    ylabel="Frequency",
    title="Final size distribution (100 runs)",
    label="Gillespie", color=:steelblue, alpha=0.7)
vline!([mean(ens.final_sizes)], label="Mean = $(round(mean(ens.final_sizes), digits=1))",
       lw=2, ls=:dash, color=:red)
```



The bimodal structure is a hallmark of stochastic epidemics near or above threshold:

- Left mode ($R_\infty \approx 0$): runs where the epidemic died out before establishing
- Right mode ($R_\infty \gg 0$): runs where a major outbreak occurred

The deterministic models predict only the major-outbreak scenario and cannot capture the extinction probability. This is one of the key reasons why stochastic validation is essential.

Summary

Aspect	Deterministic	Stochastic (Gillespie)
State variables	Continuous probabilities	Discrete counts
Trajectory	Smooth ODE solution	Step-function jumps
Extinction	Cannot model	Naturally captured
Final size	Single value	Full distribution
Computation	Fast (ODE solver)	Slower (many runs needed)

Stochastic effects matter most when:

- Population is small — fluctuations are $O(\sqrt{N})$ relative to mean
- Near the epidemic threshold — $R_0 \approx 1$ means extinction and outbreak are both likely
- Early in the epidemic — few infected individuals means high variance
- Heterogeneous networks — high-degree hubs create bottlenecks

The recommended workflow is to use deterministic models for rapid exploration of parameter space, then validate key results with Gillespie ensembles to quantify stochastic uncertainty.

NetworkOutbreaks SSA ribbon

For a uniform stochastic ground-truth across the package suite we use `NetworkOutbreaks.jl`'s Gillespie SSA. Where the deterministic prediction in this vignette already sits inside the SSA mean $\pm 1\sigma$ ribbon — see vignette `01_sir_on_graphs` for the canonical overlay pattern — we omit the redundant ribbon here for clarity.

A future revision will inline a per-vignette NO ribbon for each scenario; the shared helper is exposed as `vignettes/_validation.jl#gillespie_ribbon` and applied in vignette 01.